

Experiment # 11

Implementation of simple Vector Operations on 2D image using CUDA

Objective:

Objective of this lab is to get the knowledge about the application of basic operations on 2D-image using multiple kernel code of CUDA.

Software Tools:

- Ubuntu

Theory:

In the last lab we get familiarized with CUDA environment and implemented simple vector addition code using CUDA C language. Today by using the last lab task we update the program and perform these tasks:

- Add-Subtract in Vector element values.
- Multiply-Divide in Vector element values.
- Set Threshold to change Vector Element values.

Steps:

- Allocate and initialize array and variables on CPU
- Allocate on the GPU with cudaMalloc
- Copy from CPU to GPU with cudaMemcpy
- select which operation you want to execute (using switch-case or if-else) - Define number of blocks and threads per block
- Launch kernels (each operation has one kernel)
- Copy results to CPU with cudaMemcpy
- Print the resulting vector array.

A quick comparison between CUDA and C

Following table compares a hello world program in C and CUDA side-by-side.

C	CUDA
<pre>void c_hello(){ printf("Hello World!\n"); } int main() { c_hello(); return 0; }</pre>	<pre>__global__ void cuda_hello(){ printf("Hello World from GPU!\n"); } int main() { cuda_hello<<<1,1>>>(); return 0; }</pre>

The major difference between C and CUDA implementation is `__global__` specifier and `<<<...>>>` syntax. The `__global__` specifier indicates a function that runs on device (GPU). Such function can be called through host code, e.g. the `main()` function in the example, and is also known as "*kernels*".

When a kernel is called, its execution configuration is provided through `<<<...>>>` syntax, e.g. `cuda_hello<<<1,1>>>()`. In CUDA terminology, this is called "*kernel launch*".

For data to be accessible by GPU, it must be presented in the device memory. CUDA provides APIs for allocating device memory and data transfer between host and device memory. Following is the common workflow of CUDA programs.

1. Allocate host memory and initialized host data
2. Allocate device memory
3. Transfer input data from host to device memory
4. Execute kernels
5. Transfer output from device memory to host

Device memory management

CUDA provides several functions for allocating device memory. The most common ones are `cudaMalloc()` and `cudaFree()`. The syntax for both functions are as follow

```
cudaMalloc(void **devPtr, size_t count);
cudaFree(void *devPtr);
```

`cudaMalloc()` allocates memory of size `count` in the device memory and updates the device pointer `devPtr` to the allocated memory. `cudaFree()` deallocates a region of the device memory where the device pointer `devPtr` points to. They are comparable to `malloc()` and `free()` in C, respectively

Memory transfer

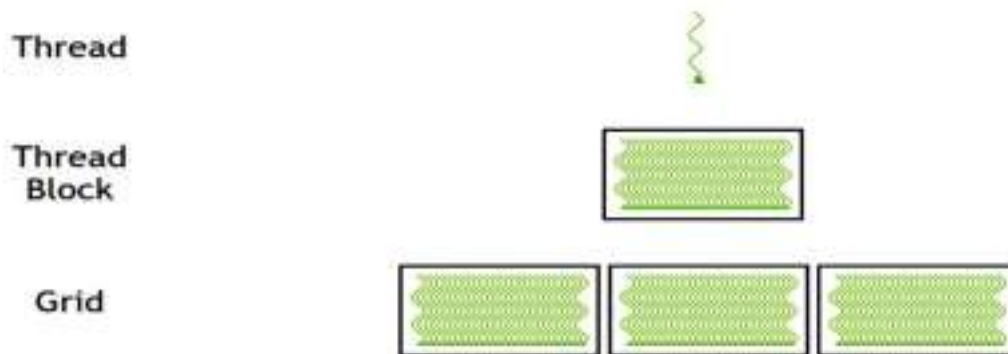
Transferring data between host and device memory can be done through `cudaMemcpy` function,

which is similar to memcpy in C. The syntax of cudaMemcpy is as follow:

cudaMemcpy(void *dst, void *src, size_t count, cudaMemcpyKind kind)

This function copy a memory of size count from src to dst. kind indicates the direction. For typical usage, the value of kind is either cudaMemcpyHostToDevice or cudaMemcpyDeviceToHost.

Thread hierarchy in CUDA C



CUDA use a kernel execution configuration <<<...>>> to tell CUDA runtime how many threads to launch on GPU. CUDA organizes threads into a group called "**thread block**". Kernel can launch multiple thread blocks, organized into a "**grid**" structure.

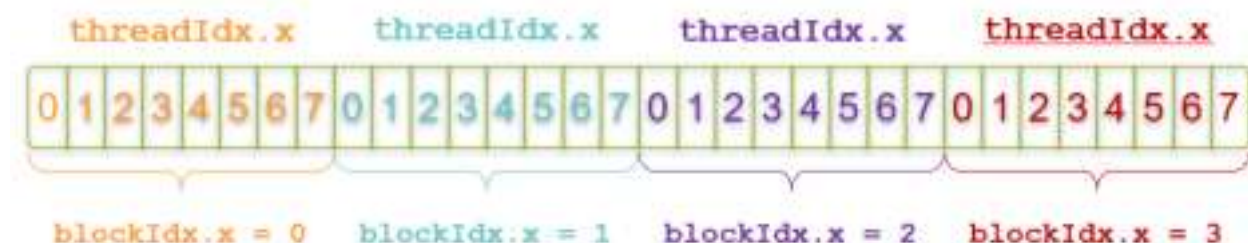
The syntax of kernel execution configuration is as follows

<<< **M**, **T** >>>

Which indicate that a kernel launches with a grid of M thread blocks. Each thread block has T parallel threads.

CUDA provides built-in variables for accessing thread information. we will use two of them: threadIdx.x and blockIdx.x.

- threadIdx.x contains the index of the thread within the block
- blockDim.x contains the size of thread block (number of threads in the thread block).



Example Code:

```
#include <cuda.h>
#include <stdio.h>

// kernel routine

__global__ void my_first_kernel(float *x)
{
    int tid = threadIdx.x + blockDim.x*blockIdx.x;

    x[tid] = (float) threadIdx.x;
}

__global__ void my_second_kernel(float *x)
{
    int tid = threadIdx.x + blockDim.x*blockIdx.x;

    x[tid] = (float) threadIdx.x + 100;
}

// main code

int main(void)
{
    float *h_x, *d_x;
    int nblocks, nthreads, nsize, n;

    // set number of blocks, and threads per block

    nblocks = 2;
    nthreads = 8;
    nsize = nblocks*nthreads ;

    // allocate memory for array

    h_x = (float *)malloc(nsize*sizeof(float)); //memory allocation on host
    cudaMalloc((void **)&d_x, nsize*sizeof(float)); //memory allocation on device

    //initialization of array
    for (int i=0; i<nsize; i++)
        h_x[i] = (float)i;
```

```
//copy addary from host to device
cudaMemcpy(d_x, h_x, nsize*sizeof(float), cudaMemcpyHostToDevice);
int ch;
printf("Enter Your Choice: ");
scanf("%d", &ch); // 0 or 1

// execute kernel
Switch(ch){
Case'0':
my_first_kernel<<<nblocks,nthreads>>>(d_x);
break;
case'1':
my_second_kernel<<<nblocks,nthreads>>>(d_x);
break;
default:
    printf("Invalid Command");
}

// copy back results and print them out

cudaMemcpy(h_x,d_x,nsize*sizeof(float),cudaMemcpyDeviceToHost);

for (n=0; n<nsize; n++)
{
printf(" h_x[%d] = %f \n", n, h_x[n]);
}

// free memory

cudaFree(d_x);
free(h_x);

}
```

Lab Task:

Assume you are given a grey scale. The **8-bits** pixel values are initialized by the random number generator in the host node having CUDA-enabled GPU-based accelerator card. Using block size of **(32 x 32)** threads, Write CUDA-C Host and Device code (including data initialization, data transfer & data processing Kernel Code) for the following image processing applications: -

a) **Brightness Measure/Control:** Measure brightness= average of pixels intensity values. Control brightness by adding/subtracting of a fixed number ($\pm b$), each pixel value. Must cater for Max-Min limits of pixel-intensity.

b) **Contrast Control:** Measure Contrast= difference of max & min pixels intensity values. Control contrast by Multiply/divide by a fixed number ($\pm c$), each pixel value. Must cater for Max-Min limits of pixel-intensity.

c) **Thresholding:** It involves setting the pixel value to **Black** if the value is less than mid-value otherwise set it to **White**, resulting in **2-level** black & white image instead of gray-level image.

In simple words:

- Write a code to generate random values in input vector limited between (0 – 255).
- Write multiple kernels for each operation i.e. add, subtract, multiply, divide & threshold.
- Using switch case statement only one kernel launch at a time.
- If add operation selected, add 80 from each element of vector. After adding if any value exceeds from 255 update it to 255.
- If subtract operation selected, minus 80 from each element of vector. After subtracting if any value became less than 0 update it to 0.
- For threshold, check each value of vector array if value is more than 120, update it to 255 else update it to 0.
- Initialize two variables which will be used for add-subtracting, Mul-divide in input vector elements and for setting threshold value to update input vector elements.

Conclusion:
